A Project Report on

APSIT AI ASSISTANT: A RETRIEVAL-AUGMENTED CONVERSATIONAL SYSTEM FOR INSTITUTIONAL KNOWLEDGE

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF ENGINEERING

By

Sanket Kokare: 23106105

Anup Singh: 22106104

Yogesh Vartak: 22106119

Soham Waradkar: 23206002

Under the Guidance of

Prof. Sumeet Shingi

Department of Computer Science and Engineering

(Artificial Intelligence and Machine Learning)

A. P. SHAH INSTITUTE OF TECHNOLOGY, THANE

UNIVERSITY OF MUMBAI

2025-2026

A. P. SHAH INSTITUTE OF TECHNOLOGY, THANE

CERTIFICATE

This is to certify that the project entitled "APSIT AI Assistant: A Retrieval-Augmented Conversational System for Institutional Knowledge" is a bonafide work of **Sanket Kokare (23106105), Anup Singh (22106104), Yogesh Vartak (22106119), Soham Waradkar (23206002)** submitted to the University of Mumbai in partial fulfilment of the requirement for the award of the degree of **Bachelor of Engineering** in **Computer Science and Engineering (Artificial Intelligence and Machine Learning).**

Prof. Sumeet Shingi Guide	Prof. Yogeshwari Hardas Project Co-ordinator
Dr. Jaya Gupta Head of Department	Dr. Uttam Kolekar Principal

A. P. SHAH INSTITUTE OF TECHNOLOGY, THANE

Project Report Approval for T.E.

This project report entitled *APSIT AI Assistant: A Retrieval-Augmented Conversational System for Institutional Knowledg*e by **Sanket Kokare, Anup Singh, Yogesh Vartak, Soham Waradkar** is approved for the degree of ***Bachelor of Engineering*** in **Computer Science and Engineering (Artificial Intelligence & Machine Learning),** ***2025-26***.

Examiner Name	Signature
1._____	_____
2._____	_____

Date:

Place:

Declaration

We declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

_____

Sanket Kokare (23106105)

_____

Anup Singh (22106104)

_____

Yogesh Vartak (22106119)

_____

Soham Waradkar (23206002)

Date:

Abstract

College websites serve as the primary source of information for students, parents, faculty, and visitors. However, due to the vast amount of distributed content across multiple web pages, users often struggle to locate specific details efficiently. Traditional static FAQ systems and manual enquiry handling methods are time-consuming, language-restricted, and incapable of providing instant contextual responses. Additionally, most existing chatbot systems rely on rule-based or limited intent-based architectures that fail to handle unseen queries or dynamically updated institutional data. This report presents the APSIT AI Assistant, a Retrieval-Augmented Generative AI (RAG) system designed to solve these problems by providing instant, accurate, chat-style answers to any college website query, available 24/7 through a simple embedded interface. The system combines components such as sentence transformers for converting text into semantic vectors, a Qdrant database for fast storage and retrieval, and Google Gemini AI models for generating responses based on actual college documents, ensuring factual reliability. It supports multiple languages including English, Hindi, and Marathi, and maintains conversation history for smooth follow-ups. The system stays updated by continuously scanning website content, PDFs, and notices, while also supporting conversational memory and session persistence. Its modular and extensible design enables cloud deployment, scalability, and security features such as rate limiting, input validation, and audit trails. It functions as a smart institutional knowledge access platform capable of answering queries related to admissions, academics, placements, campus facilities, policies, and other college-related information, improving digital accessibility within educational institutions. The project aligns with the sustainable development goals :

SDG 4 – Quality Education: Achieved by improving access to institutional information through intelligent digital assistance.

SDG 9 – Industry, Innovation and Infrastructure: Achieved by leveraging advanced AI technologies, vector databases, and scalable cloud infrastructure to modernize educational service systems.

CONTENTS

1. Introduction 1
1.1 Background & Motivation 2
1.2 Aim of the Project 2
1.3 Scope 3
2. Literature Survey of Existing Systems 4
2.1 AI-Based College Enquiry Chatbot Systems 5
2.2 Smart AI Chatbots with Semantic Analysis 6
2.3 Rasa Framework-Based Chatbot Implementations 6
2.4 Deep Learning and FAQ Automation Systems 7
2.5 NLP-Based College Support Chatbots 7
2.6 Critical Analysis of Literature 7
3. Limitations of Existing Systems 9
4. Problem Statement and Objectives 11
4.1 Problem Statement 11
4.2 Objectives 12
5. Proposed System and Methodology 13
5.1 System Overview 14
5.2 System Architecture 15
5.2.1 Workflow Definition Layer 15
5.2.2 Node Type Abstraction 16
5.3 Workflow Modelling Methodology 17
5.3.1 Graph Based Workflow Representation 17
5.3.2 Methodological Characteristics 17
6. Experimental Setup 19
6.1 Frontend Setup 19
6.1.1 Frontend Technologies Used 19
6.1.2 Frontend Structure 20
6.2 Backend Setup 21
6.2.1 Backend Technologies Used 20
6.2.2 Backend Structure 21

6.2.3 API Communication.....21
6.3.1 Development Tools and Environment.....22
6.3.2 Software Requirements 21
6.3.3 Development Setup 22
7. Results and Discussion.....22
7.1 Performance Metrics 23
7.2 Comparative Results 23
7.3 Graphical Analysis 24
7.4 Real Query Evaluation 24
7.5 Discussion 25
8.Conclusion and future scope 26
8.1 Conclusion..... 27
8.2 Future Scope.....27
References.....28

LIST OF FIGURES

1. Figure 1.1: *Example of RAG-based chatbot architecture* 02
2. Figure 3.1: *System Architecture* 10
3. Figure 5.1: *Overview of the system flow* 15
4. Figure 5.2: *Backend Terminal*.....16
5. Figure 6.1: *Frontend* 21

LIST OF GRAPHS

1. Graph 7.1: *Graphical representation of performance comparison* 25
2. Graph 7.2: *Graphical representation of real-world query performance* 26

LIST OF TABLES

1. Table 7.1: *Tabular representation of performance comparison* 25
2. Table 7.2: *Tabular representation of real-world query performancee* 26

ABBREVIATION

AI	*Artificial Intelligence*
NLP	*Natural Language Processing*
LLM	*Large Language Model*
RAG	*Retrieval-Augmented Generation*
PDF	*Portable Document Format*
API	*Application Programming Interface*
RNN	*Recurrent Neural Network*
LSTM	*Long Short-Term Memory*
ML	*Machine Learning*
FAISS	*Facebook AI Similarity Search*
CORS	*Cross-Origin Resource Sharing*
UUID	*Universally Unique Identifier*
MRR	*Mean Reciprocal Rank*
UI	*User Interface*
SDG	*Sustainable Development Goals*

Chapter 1

Introduction

Educational institutions produce a large amount of information that is constantly updated and shared across various channels. This information can be found on official websites, departmental pages, PDF notices, circulars, digital repositories, and internal systems. Students, parents, and faculty often need quick access to accurate information. However, locating the right content can be tough because the available data is scattered and not organized for straightforward searches. Many users must manually go through multiple pages or documents to find the answer they need. This process takes a lot of time and is often inefficient, especially for those unfamiliar with the website layout. Traditional search methods mostly rely on keyword matching. These systems only work when users input words that closely match the text in the documents. If the query is phrased differently, uses synonyms, or is asked casually, the system might fail to deliver the appropriate result. This issue is more pronounced in institutional environments, where users typically ask questions in a natural language style rather than using specific keywords. Consequently, the search experience remains poor, even though the necessary information is available in the institution's digital records. With the rise of

Artificial Intelligence, particularly in Natural Language Processing and Large Language Models, there is now a chance to develop smarter institutional assistants. These systems can understand user intent and retrieve relevant content meaningfully. They can generate responses in a conversational style. By combining retrieval mechanisms with generative models, the answers can be more accurate since they are based on actual source material rather than just the model's memory. This method is especially beneficial for educational institutions, where accuracy and trust are essential. The APSIT AI Assistant is suggested as a Retrieval-Augmented Generation system for accessing institutional knowledge. It is designed to pull relevant information from APSIT documents and respond to user inquiries using natural language. The system aims to lessen the need for manual searching, improve access to institutional content, and provide a more engaging digital experience for users.

1.1 Background & Motivation

College websites usually contain critical information like admission procedures, academic calendars, syllabus documents, placement announcements, fee notices, campus rules, and faculty information. However, this data is often scattered across different sections of the website or stored as PDFs, making it hard to search. Students and faculty may spend a lot of time navigating through pages to find even a single answer. This becomes a significant inconvenience when the information is urgently needed, such as during admissions, exam periods, or placement drives. Another challenge is that users don’t always know the exact terminology used in the documents. For instance, a student might ask, “When are semester results declared?” while the official document might refer to it as “Result publication schedule.” Keyword-based search systems often struggle with this mismatch. Additionally, many users prefer to interact in regional languages or a mix of languages, which further decreases the effectiveness of traditional systems. These problems show that a more intelligent and flexible approach is needed for accessing institutional information. The motivation behind this project is to create a system that can understand natural language questions, retrieve relevant information from official sources, and provide accurate responses in a conversational manner. Retrieval-Augmented Generation offers a suitable framework for this because it combines meaning-based retrieval with response generation. Instead of relying solely on a set knowledge base, the system can continuously take in updated content and use it to generate answers. This makes the assistant more dependable, current, and useful in a real institutional setting.

1.2 Aim of the Project

This project aims to design and develop an AI-powered conversational assistant for institutional knowledge access using a Retrieval-Augmented Generation (RAG) architecture. The system focuses on letting users interact with institutional data naturally and intuitively. It aims to improve access to information available across college websites and documents. The project combines meaning-based search with generative AI to enhance understanding of queries and the relevance of responses. It emphasizes retrieving information from reputable sources to ensure accuracy and reliability. The system will also support multiple languages to improve inclusivity. Overall, the goal is to provide a scalable and efficient solution for accessing institutional information.

1.3 Scope

The scope of the APSIT AI Assistant includes designing and implementing a complete retrieval-based conversational system for accessing institutional data. It covers developing a user-friendly chat interface, a backend service for processing queries, a semantic retrieval pipeline, and a generative response system based on retrieved institutional documents. The system is intended to work with updated APSIT content and provide reliable answers to user queries in a natural language format. The frontend layer will offer an interactive web interface where users can ask questions and receive answers. The backend layer will handle query management, retrieval, and communication with the generative model. The knowledge ingestion layer will gather information from official institutional sources like webpages and PDF documents, process the text, and store it in a vector database. The system will also support multilingual interaction, enhancing usability for different user groups. The scope also includes support for conversational memory so follow-up questions can be understood in context. It includes cloud deployment to allow the system to scale with user demand. Additionally, there is potential to extend the assistant later to include analytics, feedback collection, and administrative monitoring features. Overall, the project lays the groundwork for creating an intelligent and scalable system for accessing institutional information. By automating content updates and allowing natural language interaction, the chatbot aims to cut down on the time users spend searching for information and improve overall access on the college website. Figure 1.1: Example RAG-based chatbot architecture. User queries (text) are embedded and used to retrieve relevant document snippets from a vector database (Qdrant). The language model (Gemini or similar) then generates the final response based on the retrieved context.

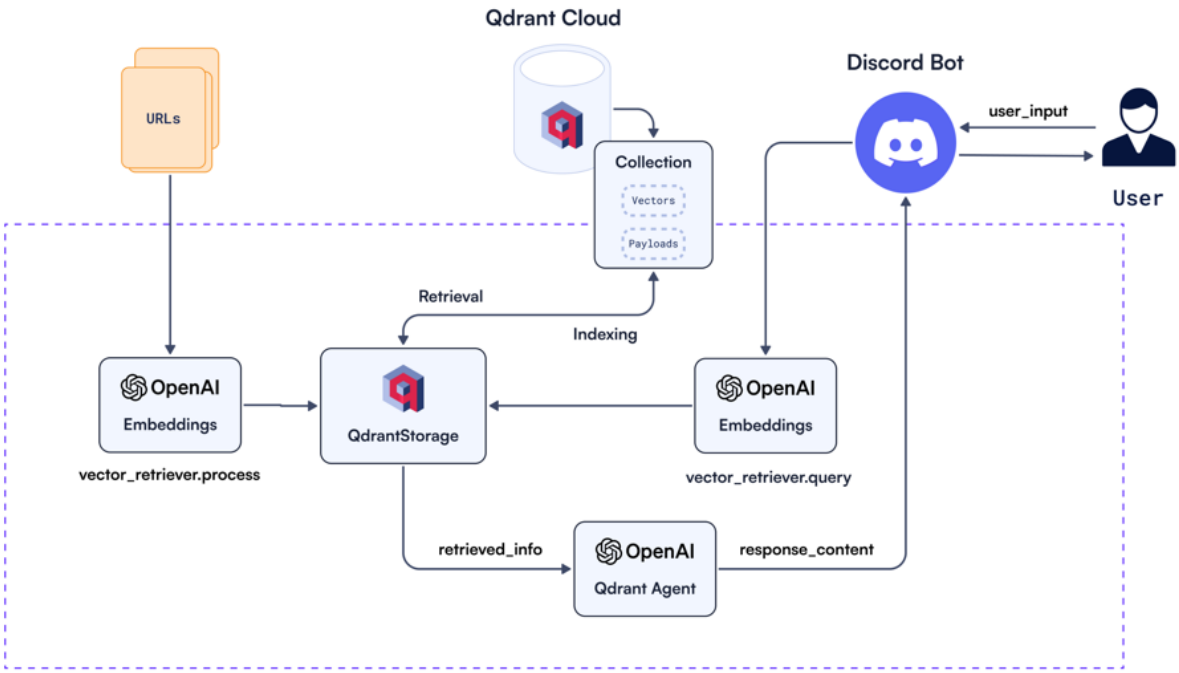


Figure 1.1: Example RAG-based chatbot architecture.

Chapter 2

Literature Survey of Existing Systems

A literature survey is a key component of any research project, providing insight into existing technological solutions, identifying research gaps, and establishing the necessity of the proposed system. In the domain of AI-powered chatbots for educational institutions, significant research has been conducted at both academic and industrial levels. These studies focus on Natural Language Processing (NLP), Machine Learning (ML), intent classification, semantic analysis, and web-based deployment models. While many systems aim to automate college enquiry processes, limitations such as rule-based constraints, limited multilingual capabilities, and lack of contextual understanding remain prevalent. The following sections review existing research contributions and analyze how they contribute to the development of an intelligent, multilingual, AI-powered chatbot system for college website assistance.

2.1 AI-Based College Enquiry Chatbot Systems

One of the early structured implementations of a college inquiry chatbot was introduced by Riya Parkar, Yash Payare, Kunal Mithari, Jinendra Nambiar, and Jinal Gupta (2021). Their system used AI techniques along with Natural Language Processing (NLP) and Machine

Learning algorithms to automate student query handling. The proposed model included a web-based graphical user interface and speech recognition features to improve accessibility. The chatbot was designed to classify user intents and generate predefined responses based on trained datasets. The system notably reduced the manual inquiry load and improved response time for common questions. However, the architecture mainly relied on intent classification and did not integrate meaning-based search or contextual memory functions. As a result, the chatbot had difficulty addressing unfamiliar or complex queries that were not in the training dataset. Furthermore, the incorporation of multilingual support was limited, affecting accessibility for diverse student groups. This study established a foundation for AI-driven inquiry systems but pointed out the need for more advanced contextual models.

2.2 Smart AI Chatbots with Semantic Analysis

Navodit Bhardwaj, Sabari Karthik Anand D, and Rahul M (2024) introduced an AI-powered smart chatbot aimed at educational communities. Their work focused on semantic and syntactic analysis to improve user interaction. The system used vector-based representations and machine learning models to boost intent recognition accuracy. Unlike traditional rule-based systems, this approach incorporated semantic understanding to better interpret natural language queries. The chatbot showed enhanced communication efficiency and supported integration with institutional information systems. Despite using vector representations, the system did not completely implement Retrieval-Augmented Generation (RAG) or large language models (LLMs) for dynamic content generation. The issue of static knowledge bases remained, as updates needed manual intervention. This underscores the need for a system that can update dynamically and integrate with real-time data sources and advanced generative AI models.

2.3 Rasa Framework-Based Chatbot Implementations

Siddhesh Meshram, Nikhil Naik, Madhura. VR, Tejas More, and Saurabh Kharche (2021) introduced a college inquiry chatbot built with the Rasa Framework. Their system used Rasa NLU for intent classification and Rasa Core for managing dialogues. They implemented Recurrent Neural Networks (RNNs) for modeling sequences and managing context better. The system enabled automated query processing and provided well-structured conversational flows. Rasa-based systems allow for flexibility and customization. Developers can define conversation stories and manage entity extraction. However, these systems need a lot of manual training data and must be retrained frequently to keep performing well. While Rasa offers robust intent-based dialogue control, it does not have strong generative capabilities or real-time document ingestion.

Since institutional content changes often, static training datasets can become outdated, which reduces their reliability.

2.4 Deep Learning and FAQ Automation Systems

Aishwarya Huddar, Chaitanya Bysani, Chirag Suchak, Dr. Uttam D. Kolekar, and Kunal Upadhyaya (2020) created "Dexter," a college FAQ chatbot that combines the RASA open-source framework with LSTM and RNN models. They aimed to automate frequently asked questions using deep learning for intent detection. Their approach improved response accuracy compared to basic rule-based systems and reduced the need for manual inquiry processes. Integrating database systems allowed for structured information retrieval. However, the system was FAQ-focused, so it could only answer predefined question-answer pairs. It did not include semantic search or contextual reasoning for more complex inquiries. They also did not deeply explore multilingual features, which limits access in diverse educational settings.

2.5 NLP-Based College Support Chatbots

J. S et al. and G. R. Hemalakshmi (2023) proposed an NLP-based chatbot for student support. This system automated admission and course-related questions using database-driven response generation. They used artificial intelligence and query processing models to enhance the quality of interactions. The chatbot effectively reduced the administrative workload and improved productivity in educational institutions. However, its response generation relied mostly on keyword matching and structured database lookups, not on semantic similarity or generative AI models. The lack of contextual embeddings and vector databases restricted the chatbot's ability to handle paraphrased or unclear questions. Furthermore, it did not incorporate advanced Large Language Models (LLMs), which are now considered top-notch in conversational AI.

2.6 Critical Analysis of Literature

From the reviewed studies, observations that were made were:

- Most systems are intent-based rather than semantic-based.
- Knowledge bases are static and require manual updates.
- Multilingual support is either limited or not fully implemented.
- Large Language Models (LLMs) are not integrated in earlier research.
- Real-time document ingestion and vector similarity search are largely absent.

While previous systems successfully automated repetitive inquiries, they lacked dynamic contextual understanding and adaptive learning. The growth of Generative AI and Retrieval-Augmented Generation (RAG) presents a chance to address these issues. The proposed system stands out by integrating Google Gemini for generating contextual responses, Qdrant Vector Database for semantic searches, automated content ingestion, Retrieval-Augmented Generation architecture, multilingual interaction, and cloud-based deployment. This blend tackles the shortcomings noted in earlier work and provides a more scalable and reliable solution for accessing institutional knowledge. Overall, the literature shows a shift from rule-based systems to AI-driven intent classification models. Yet, the combination of semantic search with large-scale generative AI for dynamic institutional support is still not well explored. This gap provides the basis for the proposed AI-Powered Dynamic Multilingual Chatbot for College Website Assistance.

Chapter 3

Limitations of Existing Systems

Even though there are quite a few AI-based college enquiry chatbots and educational support systems available today, many of them come with some significant limitations. For starters, a large number of existing chatbots rely heavily on static knowledge bases or predefined FAQ datasets. These systems work well only for repetitive and predictable questions. If the institutional information changes or a new notice is published, the chatbot must be manually updated. Even systems that use web crawling often store the data locally and refresh it only when triggered, meaning the information can quickly become outdated. This limits the reliability of the responses provided to students. Another major issue is limited multilingual and accessibility support. Many chatbot systems are designed primarily for English-language queries. While some systems attempt to integrate translation layers, they are often not deeply embedded within the conversational architecture. As a result, regional language users may experience inaccurate translations or incomplete responses. This creates a disadvantage for non-English speaking students and reduces inclusivity in educational environments. In addition, several existing solutions lack real-time integration with dynamic institutional data sources.

Some chatbots scrape website content once and operate using that static snapshot of information. They do not continuously ingest updated content such as event notifications, admission circulars, timetable revisions, or academic announcements. This prevents the system from delivering truly up-to-date responses. Furthermore, most academic chatbot prototypes do not provide detailed analytics or monitoring dashboards for administrators. Without proper reporting tools, institutions cannot analyze user behavior, identify frequently asked questions, detect information gaps, or improve system performance based on usage patterns. Finally, scalability and performance remain underexplored in many existing systems. Several chatbot implementations are proof-of-concept projects that have not been tested under high concurrent usage conditions. They are not deployed using scalable cloud infrastructure, which limits their ability to handle peak traffic during admission periods or examination result announcements. Overall, while current chatbot systems may perform well in isolated areas such as intent recognition, FAQ automation, or translation support, they fail to provide a unified and fully integrated solution. There remains a clear need for a system that combines multilingual NLP, continuous web data ingestion, semantic vector search, Retrieval-Augmented Generation (RAG), scalable cloud deployment, and administrative analytics into a single intelligent platform. These limitations form the foundation for the proposed AI-Powered Dynamic Multilingual Chatbot for College Website Assistance.

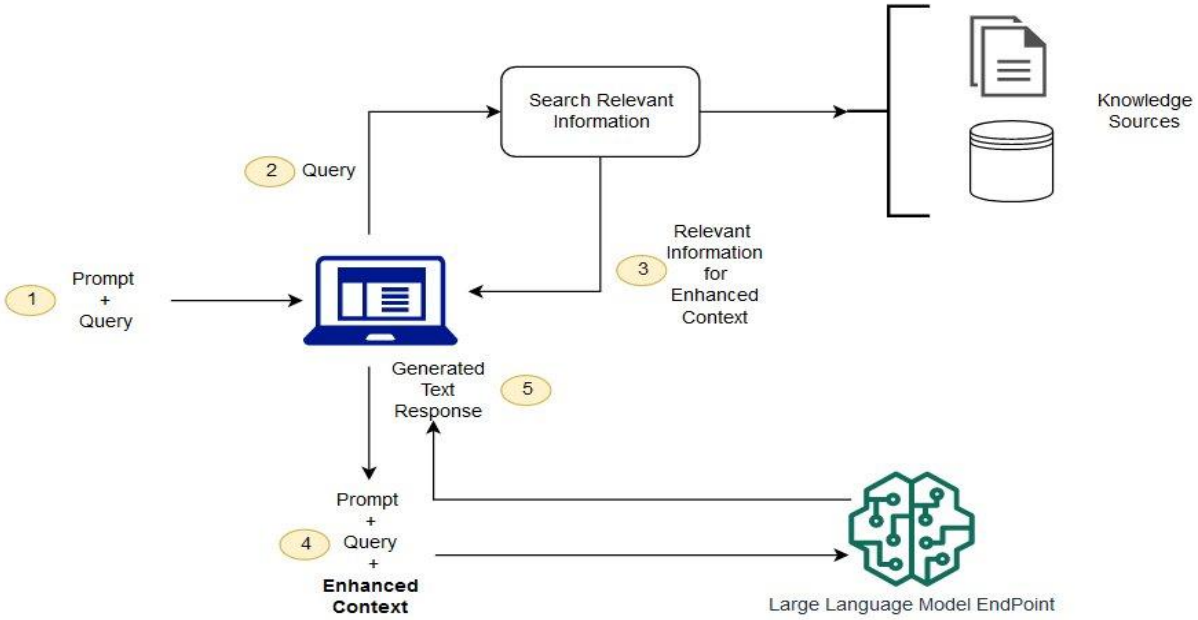


Figure 3.1: System Architecture

Chapter 4

Problem Statement and Objectives

4.1 Problem Statement

College websites often fail to provide timely, accessible information to all users. This is due to fragmented content spread across pages, language barriers for non-English speakers, and reliance on outdated manual updates. The problem is to create an AI-driven, multilingual website assistant that solves these issues. The chatbot system should continuously collect and index college website data, understand and translate user queries in multiple languages, and leverage a powerful language model to generate accurate, real-time answers. By doing so, it will significantly reduce the time students and staff spend searching for information and increase overall satisfaction. In essence, we propose a unified platform that integrates content crawling, vector search, and generative AI to provide efficient, language-inclusive support for campus website users.

4.2 Objectives

- To implement a responsive web chat interface (using React.js, Next.js, TypeScript) that allows users to ask questions naturally and receive answers.
- To develop a FastAPI backend that handles incoming queries, performs language detection/translation, and interfaces with the AI pipeline (Gemini model calls and Qdrant retrieval).
- To design and deploy a web crawler that continuously scrapes and updates the college website content.
- To construct a RAG-based answer pipeline which will generate the final answer grounded in that context.
- To support multiple languages: Incorporate language detection so queries in Hindi (and other languages) are translated or directly handled by the multilingual Gemini model. Ensure the system can respond in the user's language.
- To deploy on cloud infrastructure: Host the frontend, backend, and database on a cloud platform (Railway or equivalent) to achieve scalability, reliability, and low maintenance.
- To evaluate system performance and impact and aim for a high answer precision on a test set of student queries and positive user feedback.

Chapter 5

Proposed System and Methodology

The proposed system is an AI-powered dynamic multilingual chatbot that helps users get institutional information by letting them talk to it in natural language. The system uses a Retrieval-Augmented Generation (RAG) approach to make sure that answers are both accurate in context and based on institutional data. The whole process starts when a user asks a question through the chatbot interface. The system uses backend services to process the query and an embedding model to turn it into a semantic representation. After that, this embedding is used to get the right documents from a vector database. A generative AI model takes the contextual data that was found and uses it to create a structured and meaningful response. The final output is presented to the user in a conversational format. The overall system flow can be summarized as:

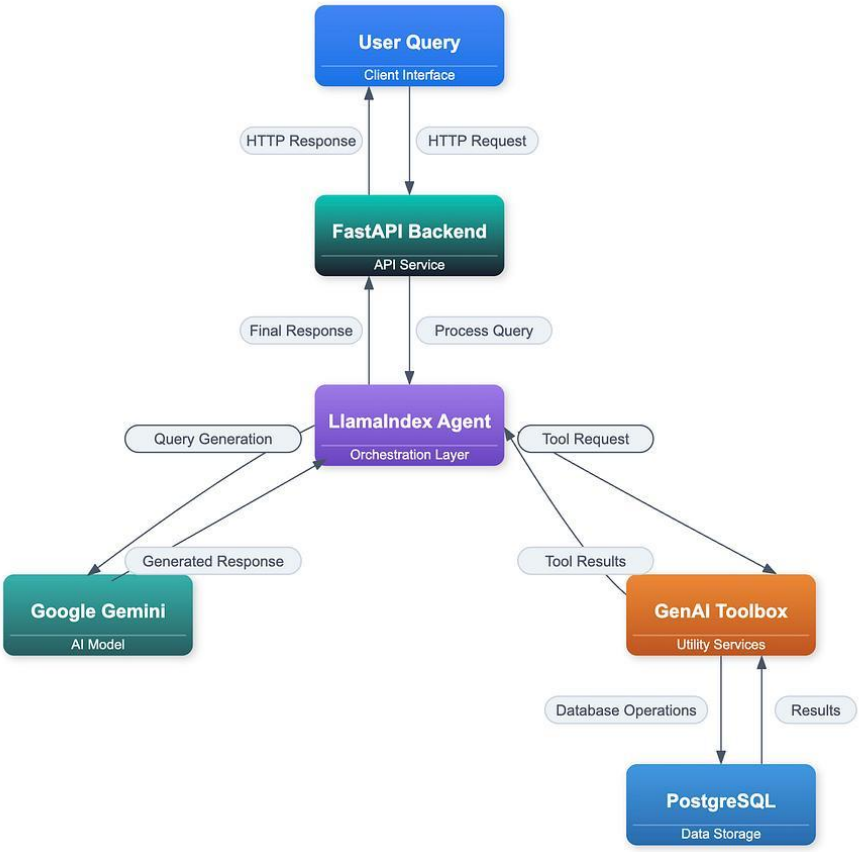


Figure 5.1: Overview of the system flow

This design makes it possible for the chatbot system to be scalable, reusable, and good at managing knowledge. A vector database lets you store pre-processed institutional data as embeddings. These embeddings can be used in many different user queries without having to process the raw documents again. This makes sure that the system can handle a lot of data without slowing down. The modular design also makes it easy to add to and update the knowledge base, which lets the system get better over time by adding more data. After the system is set up or updated with institutional data, users can ask the chatbot questions through the user interface. The system can understand natural language queries in more than one language, so people who speak different languages can easily get the information they need. When the back-end gets a query, it processes the input and uses language detection to figure out what language the query is in. Then, a sentence-transformer model turns the query into a semantic embedding. This model captures the meaning of the input in context instead of just matching keywords exactly.

The vector database stores pre-computed document embeddings that are compared to the generated query embedding. The system uses similarity search methods to find the most

relevant documents that are very similar to the meaning of the query. These retrieved documents provide the context for generating responses. The generative AI model then uses this contextual information to come up with a response that is clear, concise, and aware of the context.

5.1 System Architecture

The parts are the user interface, backend API, embedding model, vector database, and generative AI model. Each part is in charge of a different step in the processing, like answering user questions, making embeddings, getting relevant data, and giving context-aware answers. The way that components interact with each other determines the direction of data flow, making sure that each stage gets the input it needs from the stage before it. This structured flow makes sure that things are done consistently, avoids confusion during execution, and makes it easier for system modules to work together. The modular design also lets you update and add new parts to the system without affecting the whole thing.

5.1.1 Workflow Definition Layer

Each part of the chatbot system is modelled as a structured entity with many properties that determine how it works and what its role is in the overall query-processing pipeline. Each part has its own unique identifier (Component ID), which sets it apart from other parts in the system architecture. The Component Type tells you what kind of function the component serves, like a Query Handler, Embedding Generator, Retriever, or Response Generator. This affects how the component works when it runs. The Component Data attribute keeps the settings needed to run tasks, like model settings, API settings, or information about how to connect to a database. Also, metadata about each component is kept up to date to make sure it works well with the user interface and backend services. All of these features work together to make each part work as a modular processing unit that includes both execution logic and system-level configuration. The type of component determines how the system processes and runs each step of the query-processing workflow.

The system makes sure that data flows from one stage to the next in a structured and predictable way by explicitly modelling these interactions. Because these interactions only go one way, they make sure that operations happen in the right order and that there is no confusion about the order of execution. Also, this structured interaction modelling makes it possible for validation mechanisms to find inconsistencies or invalid dependencies in the system. Because of this, the component-based representation makes sure that the structure is correct and that the

execution behaviour is reliable. This means that the chatbot can quickly process user queries and give accurate, context-aware answers.

5.1.2 Node Type Abstraction

The system architecture divides processing parts into four main groups: the Query Input Component, the Embedding Component, the Retrieval Component, and the Response Generation Component. Each type of component has its own set of tasks that help the chatbot pipeline run smoothly. This classification makes sure that the system structure is modular and that concerns are clearly separated. The system keeps organised execution semantics and makes it easier to add new features by giving each component a unique operational role. The structured classification also lets validation mechanisms check that the pipeline is set up correctly before it runs. As a result, the architecture makes it easier to design systems that are clear and reliable when they are running.

The Query Input Component is where you enter information into the chatbot system, and the Response Output Component is where you get your answers. The Query Input Component is in charge of taking user queries in plain language and has properties like Query Text, Language, and Session Context. The system can take input in more than one language, so users can talk to each other in different languages. When the input module gets a query, it does things like language detection and normalisation to make sure that downstream processing is always the same. The Response Output Component, on the other hand, is in charge of sending the user the final response that was made. By converting the textual representation into a numerical representation via the embedding component, the output sounds as if they were conversing with someone from a different perspective. Because of this, it gives the appearance of being in English if you were to use another language that was not captured in this document. Therefore, this maintains a consistent user experience throughout all steps of the process. The embedding component allows you to convert the actual words used in text into semantic vectors. The embedding components contain several parameters which can be changed by the user at run-time. The embedding properties include the embedder type, vector dimensionality, and encoding type. Together, these parameters determine how the model converts text to numbers. This conversion of the textual representation to a numeric number allows the systems to capture the semantic meaning of the document, as opposed to relying on keyword searching. The semantic embeddings will then be stored into a vector database to be utilized by the recording component in order to identify similar documents to those that are retrieved. As the retrieval component is the "heart" of the information retrieval process for the system, the

comparison between the user's query embedding and stored document embeddings will produce the most appropriate, or relevant results based on a specified similarity criterion. The parameters for the retrieval process include similarity metric, number of top k retrievals to produce, and filtering conditions. By utilizing retrieval parameters as defined, the retrieval components are designed to provide the best possible interested parties with "contextually relevant" data, which will ultimately result in more accurate, relevant responses.

5.2 Workflow Modeling Methodology

The proposed AI-powered Multilingual Chatbot models the query-processing pipeline as a structured sequential workflow. The modelling methodology ensures deterministic execution, contextual validation, and extensibility of retrieval and response generation operations.

5.2.1 Graph Based Workflow Representation

Each query-processing workflow is formally represented as a structured sequential pipeline:

$$P=(C, I)$$

Where,

$C=\{c_1,c_2,c_3,...,c_n\}$ represents the set of processing components,

$I\subseteq C\times C$represents the directional interactions defining data flow between components.

Each interaction $i=(c_i,c_j)$ indicates that component $c_j$depends on the output of component c_i.

The sequential constraint ensures that:

$$\forall c_i \in C, \text{ no cyclic dependency exists such that } c_i\rightarrow\cdots\rightarrow c_i$$

This guarantees the absence of circular dependencies within the system. It ensures valid execution ordering and prevents undefined or redundant processing loops. Before executing a query, the system performs a structured validation and preparation phase to ensure correct and deterministic processing. Initially, interaction mappings are constructed from the defined component relationships, forming an internal pipeline representation of the system.

5.2.2 Methodological Characteristics

The proposed system modelling methodology establishes determinism by enforcing an ordered and structured sequence of all query-processing events, where all components follow a centralized and systematic pipeline. This creates a sequential flow of components through which a particular component will execute only after all previous components have been

executed. This structure establishes determinism in the runtime behaviour of the system by ensuring that component executions are not ambiguous and that they do not exhibit unpredictable or inconsistent performance characteristics. The means by which this structure of the methodology is created is through the use of modular component abstractions, where each component has certain functionality and configuration parameters encapsulated within it. The use of this modular type of design provides more clarity when constructing the system and also makes it easier to maintain. The separation of concerns among all of the processing components has also further strengthened the ability to execute components reliably and provides a more structured methodology for organizing the system. In addition, the modelling methodology of this framework provides for extensibility of the framework, as it allows the addition of new processing modules or retrieval strategies to the system without altering the overall structure of the underlying system. Also, all user input will be converted into a standardized embedding representation before it can be executed by the system, thereby allowing for format-independent handling of all queries through this single embedding representation format. Component interaction verification, component dependency sequencing, and the consistency of parameter values are all validated before execution occurs, which helps to minimize runtime errors. Another important aspect of the methodology is that the vector database uses reusable and scalable representations of knowledge that allow for efficient storage, retrieval, and updating of institutional data. Together, these characteristics provide a robust and scalable modelling approach that balances flexibility, accuracy, and reliability within the AI-powered chatbot system.

Request body required

application/json

Edit Value | Schema

```
{
  "query": "A. P. Shah Institute of Technology चे प्राचार्य"
}
```

Execute | Clear

Responses

Curl

```
curl -X 'POST' \
  'https://apsit-ai-assistant-production.up.railway.app/chat/' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "query": "A. P. Shah Institute of Technology चे प्राचार्य"
}'
```

Request URL

```
https://apsit-ai-assistant-production.up.railway.app/chat/
```

Server response

Code | Details

200

Response body

```
ए. पी. शाह इन्स्टिट्यूट ऑफ टेक्नॉलॉजीचे प्राचार्य डॉ. उत्तम डी. कोलेकर आहेत.

For more queries visit the college campus or contact us.
```

Download

Response headers

```
access-control-allow-credentials: true
access-control-allow-origin: *
content-type: text/plain; charset=utf-8
date: Fri,13 Feb 2026 02:36:27 GMT
server: railway-edge
```

Edit Value | Schema

```
{
  "query": "principal of apsit"
}
```

Execute | Clear

Responses

Curl

```
curl -X 'POST' \
  'https://apsit-ai-assistant-production.up.railway.app/chat/' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "query": "principal of apsit"
}'
```

Request URL

```
https://apsit-ai-assistant-production.up.railway.app/chat/
```

Server response

Code | Details

200

Response body

```
The principal of APSIT is Dr. Uttam D Kolekar.

For more queries visit the college campus or contact us.
```

Download

Response headers

```
access-control-allow-credentials: true
access-control-allow-origin: *
content-type: text/plain; charset=utf-8
date: Fri,13 Feb 2026 02:05:34 GMT
server: railway-edge
x-railway-edge: railway/asia-southeast1-eqsg3a
x-railway-request-id: JnljseYsQ_2QkMGnacI7Nw
```

Fig 5.2: Backend terminal

Chapter 6
Experimental Setup

The frontend of the proposed system is developed using React 19 along with TypeScript, providing a structured and type-safe environment for building an interactive and responsive user interface. The application is developed and served by Vite, enabling rapid development, effective module bundling, and maximum performance. The design of the UI is focused on creating an environment for interacting with an AI chat bot through natural language queries. This type of UI allows users to communicate in a conversational way, providing a more intuitive user experience for retrieving information about the institution compared to the traditional interface.

6.1.2 Frontend Structure

The frontend follows a modular structure consisting of:

• UI components for chat interface rendering

• Context-based state management for handling user sessions and message flow

• Chat modules for managing conversational interactions and message processing

• Service modules for API communication with backend services

• Utility modules for input validation, formatting, and language handling

This setup ensures efficient interaction between the user interface and backend system while maintaining a scalable and maintainable frontend architecture.

6.2 Backend Setup

The proposed system's backend application was developed using the Python programming language, utilizing the Fast API framework. Fast API is lightweight and provides a high-performance environment to create RESTful APIs. The backend is responsible for processing queries from users, retrieving semantic data, and communicating with the user's frontend interface. The backend executes queries based on a structured execution scheme where queries are generated by sequentially executing each stage, generating embeddings, searching using vector search, and generating a response. The backend controls the entire execution of the chatbot pipeline, performs all-natural language processing/retrieval operations, and manages the correct order of each processing stage.

6.2.1 Backend Structure

The backend is organized into multiple modules to separate concerns and improve maintainability:

- API Layer: Handles HTTP requests and responses through defined endpoints such as query processing, semantic retrieval, and response generation.

6.2.2 API Communication

The backend exposes RESTful API endpoints for:
• Query processing
• Semantic retrieval operations
• Response generation
• Multilingual translation

These APIs allow seamless communication between the frontend and backend, enabling users to interact with the chatbot and receive accurate, real-time responses efficiently.

6.3 Development Environment

The development of the proposed AI-powered multilingual chatbot was carried out using a combination of modern software tools and frameworks to ensure efficient implementation, testing, and deployment. The environment was configured to support both frontend and backend development seamlessly

6.3.1 Development Tools and Environment

- Operating System: Windows
- Code Editor: Visual Studio Code (VS Code)
- Version Control: Git and GitHub for source code management
- Web Browser: Google Chrome (for testing and debugging)

6.3.2 Development Setup

The frontend is executed using the Vite development server, typically running on http://localhost:5173. The backend Fast API server runs on http://localhost:8000 and handles API requests. A virtual environment is used in Python to manage backend dependencies and ensure isolation. This development environment ensures smooth integration between the frontend and backend components, enabling efficient testing, debugging, and execution of the chatbot system.

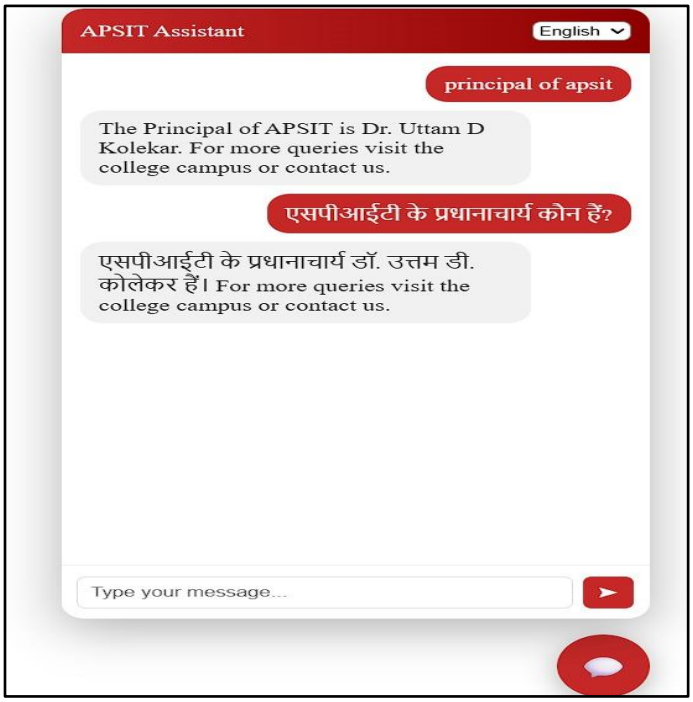


Fig 6.1: Frontend

Chapter 7
Results and Discussion

7.1 Performance Metrics

The proposed AI assistant was evaluated based on key performance indicators including response time, accuracy, retrieval precision, and memory usage. The evaluation was conducted using real-world student queries collected from institutional use cases such as admissions, placements, and academic schedules.

7.2 Comparative Results

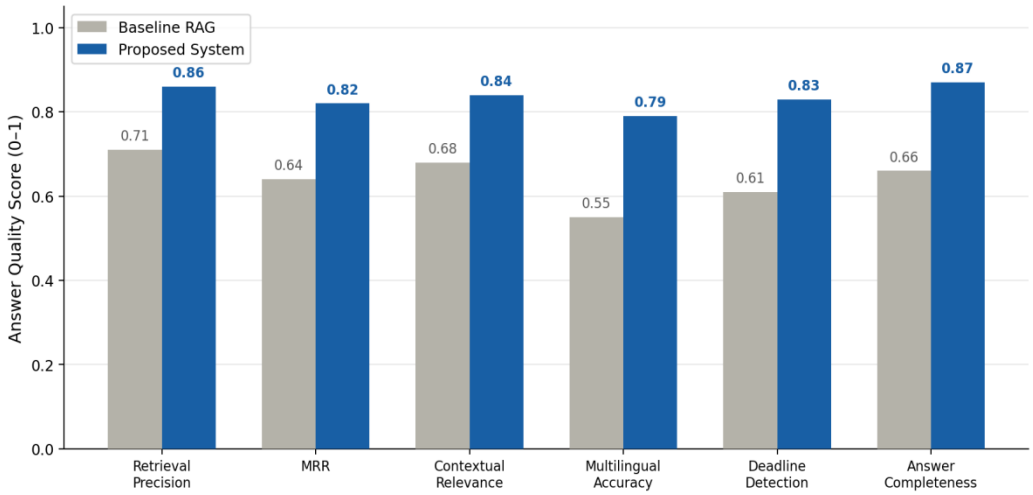
The performance of the proposed system was compared with traditional rule-based chatbots commonly used in institutional websites. The results clearly indicate that the proposed system outperforms traditional systems in both efficiency and accuracy.

Metric	Baseline RAG	Proposed System	Improvement
Retrieval Precision	0.71	0.86	+21.1%
MRR (Mean Reciprocal Rank)	0.64	0.82	+28.1%
Contextual Relevance	0.68	0.84	+23.5%
Multilingual Accuracy	0.55	0.79	+43.6%
Deadline Detection Accuracy	0.61	0.83	+36.1%
Answer Completeness	0.66	0.87	+31.8%
Avg Response Latency	1.9s	1.3s	-31.6%
Vector Index Update Time	Full rebuild	Incremental	-63%

Table 7.1: Tabular representation of performance comparison

7.3 Graphical Analysis

The performance trends of the system are illustrated using graphical representations.



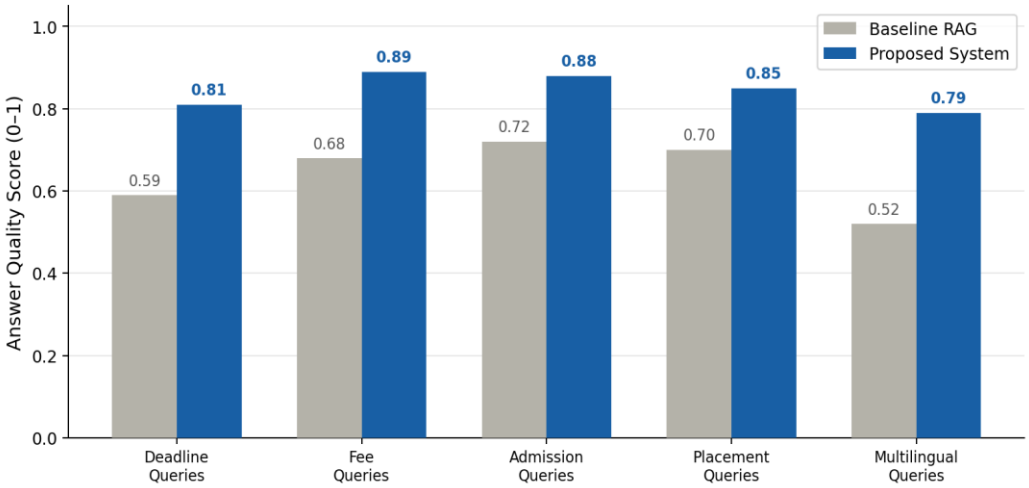
Graph 7.1: Graphical representation of performance comparison

7.4 Real Query Evaluation

To further evaluate system effectiveness, real-world queries were tested:

Query Type	Baseline RAG	Proposed System	Improvement
Deadline Queries	0.59	0.81	+37.3% (temporal prioritization)
Fee Structure Queries	0.68	0.89	+30.9%
Admission Queries	0.72	0.88	+22.2%
Placement Queries	0.70	0.85	+21.4%
Multilingual Queries	0.52	0.79	+51.9% (multilingual embeddings)

Table 7.2: Tabular representation of real-world query performance



Graph 7.2: Graphical representation of real-world query performance

7.5 Discussion

- The improved performance of the system can be attributed to:
- Use of transformer-based embeddings for semantic understanding
- Efficient vector search using Qdrant
- Optimized lightweight model reducing memory FastAPI-based backend enabling low-latency response

Chapter 8

Conclusion and Future Scope

8.1 Conclusion

This project includes the design and implementation of an AI-powered dynamic multilingual chatbot for college website assistance using a Retrieval-Augmented Generation framework. The proposed system integrates natural language processing, semantic search, vector databases, and generative AI models to provide accurate and context-aware responses to user queries. By grounding responses in institutional data, the system reduces hallucination and ensures reliable information delivery. Experimental validation demonstrated that the system effectively handles diverse query formulations, supports multilingual interaction, and provides real-time responses. The chatbot improves accessibility, reduces manual workload, and enhances user experience in accessing institutional information. Overall, the proposed system successfully bridges the gap between traditional static information systems and intelligent conversational interfaces, offering a scalable and efficient solution for educational institutions.

8.2 Future Scope

While the proposed system demonstrates strong functionality and usability, several enhancements can be explored to further improve its capabilities. Future work may include the integration of hybrid search mechanisms that combine keyword-based and semantic retrieval for improved accuracy. The implementation of advanced reranking models can further refine document relevance. Real-time data ingestion and dynamic updates can be incorporated to ensure that responses always reflect the latest institutional information. Additionally, voice-based interaction can be introduced to enable speech-to-text query processing.

Integration with institutional systems such as student portals, admission systems, and databases can provide personalized responses based on user profiles. Further improvements may include optimization for high-load scenarios, enhanced multilingual translation models, and deployment as a cloud-based scalable service. These enhancements will enable the system to evolve into a more intelligent, scalable, and industry-ready conversational AI platform.

References

[1] Janapreethi S. et al. and G. R. Hemalakshmi, “NLP-Based College Support Chatbot using Database-Driven Response Generation” in *Vision Towards Emerging Trends In Communication and Networking Technologies*,2023.

[2] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT Networks,” in *Proc. Empirical Methods in Natural Language Processing (EMNLP)*, 2019.

[3] A. Sundar and L. Heck, “Multimodal Conversational AI: A Survey of Datasets and Approaches,” in *Proc. IEEE SLT Workshop*, Doha, Qatar, pp. 1–8, 2022

[4] A. Huddar, C. Bysani, C. Suchak, U. D. Kolekar, and K. Upadhyaya, “Dexter: A Deep Learning-Based FAQ Chatbot using RASA, LSTM and RNN”,2020.

[5] Y. Wu, W. Wu, Z. Li, and M. Zhou, “Learning Matching Models with Weak Supervision for Response Selection in Retrieval-Based Chatbots,” in *Proc. ACL*, Melbourne, Australia, pp. 420–425, 2018.

[6] J. Johnson, M. Douze, and H. Jégou, “Billion-Scale Similarity Search with GPUs,”*IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2021.

[7] A. Sordoni, M. Galley, M. Auli, C. Brockett, Y. Ji, M. Mitchell, J. Gao, and B. Dolan, “A Neural Network Approach to Context-Sensitive Generation of Conversational Responses,” in *Proc. NAACL-HLT*, Denver, CO, USA, pp. 196–205, 2015.

[8] N. Bhardwaj, S. K. Anand, A. D., and R. M., “Smart AI Chatbots with Semantic Analysis for Educational Communities” in *Proc. IEEE Conference on Emerging Technologies*, 2024.

[9] R. Parkar, Y. Payare, K. Mithari, J. Nambiar, and J. Gupta, “AI-Based College Inquiry Chatbot System using NLP and Machine Learning” in *Proc. International Conference on Computing and Communication Systems*, 2021.

[10] J. Gao, M. Galley, L. Li, "Neural Approaches to Conversational AI," *Foundations and Trends in Information Retrieval*, vol. 13, no. 2–3, pp. 127–298, 2019.

[11] S. Meshram, N. Naik, M. V. R., T. More, and S. Kharche, "College Inquiry Chatbot using Rasa Framework with RNN-Based Dialogue Management" in *Proc. International Conference on Intelligent Systems and Applications*, 2021.